

CP395 Progress Report 1: Implementation & Feasibility Milestone

Sufiya Rahemtulla
Student ID: 169018559

February 15 2026

Abstract

This report outlines the implementation and feasibility milestone for developing a proactive, reliability-aware cloud autoscaler. It details the ingestion and characterization of the Google Cluster-Usage Traces v3, the establishment of reactive and predictive baselines, the formal Markov Decision Process (MDP) problem formulation, and the development of a reproducible OpenAI Gym experiment harness. Initial evaluations demonstrate the fundamental limitations of standard reactive policies and statistical forecasting when faced with heavy-tailed, bursty workloads, validating the need for the proposed Reinforcement Learning (RL) approach.

1 Problem Definition & Literature Gaps

Cloud autoscaling aims to balance system performance with resource efficiency [2]. A systematic review of recent literature (2020-2026) reveals that industry-standard reactive autoscalers (e.g., Kubernetes Horizontal Pod Autoscaler) suffer from significant “provisioning lag,” failing to scale resources in time for sudden traffic bursts [3]. While predictive Machine Learning (ML) models attempt to address this by forecasting demand, they often rely heavily on the assumption that past patterns will repeat and are vulnerable to concept drift [3]. Furthermore, existing literature often lacks realistic constraints, such as provisioning delays, and over-relies on optimizing for prediction accuracy (e.g., Mean Squared Error) rather than actual Service Level Agreement (SLA) violations [3].

The objective of this research is to bridge this gap by developing a proactive autoscaler that prioritizes reliability. This will be achieved using a hybrid approach: a supervised learning model for short-term forecasting coupled with a Reinforcement Learning (RL) agent to dynamically manage risk [3].

2 Workload Characterization & Pipeline

2.1 Dataset Selection and Ingestion

The project utilizes the Google cluster-usage traces v3, which captures realistic, large-scale workload variability over the month of May 2019 [1, 2]. The raw trace data was resampled into 5-minute intervals to align with standard cloud monitoring tools and Kubernetes HPA reaction windows [4].

Exploratory Data Analysis (EDA) revealed that the workload’s CPU usage does not follow a smooth pattern but exhibits sharp, jagged spikes, as shown in Figure 1 [4].

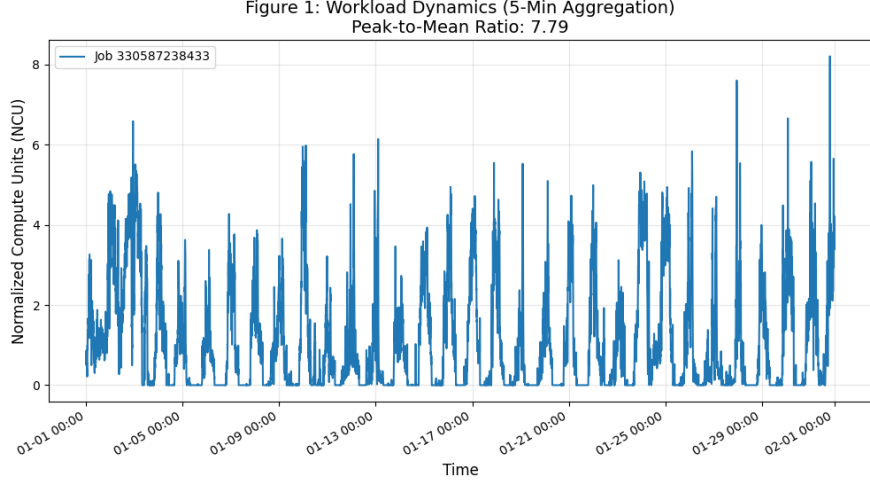


Figure 1: Workload dynamics displaying sharp, jagged CPU spikes with a peak-to-mean ratio of 7.79.

Furthermore, the probability density function (Figure 2) demonstrates a strictly “Heavy-Tailed” (right-skewed) distribution, where usage is usually low but extreme tail events occasionally occur [4]. This confirms that optimizing an autoscaler for the “average” case is dangerous, as it systematically under-provisions during critical tail events [4].

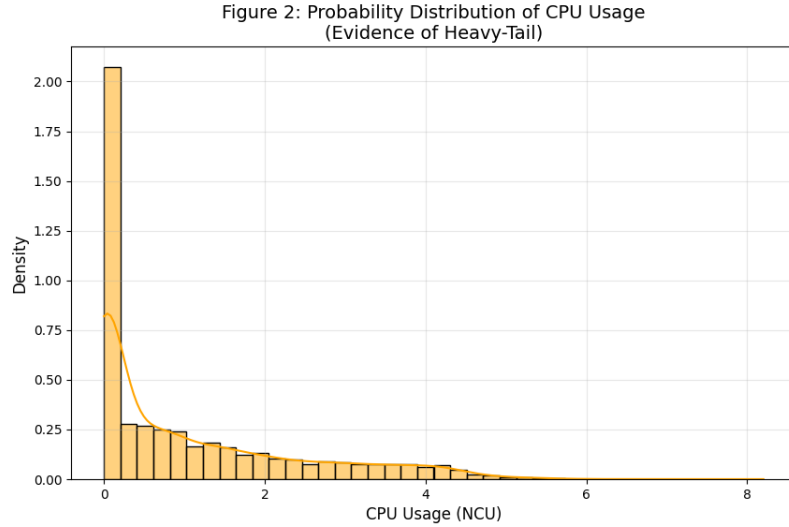


Figure 2: Probability density of CPU usage demonstrating a right-skewed, heavy-tailed distribution.

2.2 Workload Synthesis

To improve generalizability and ensure realistic stress testing, a “Workload Synthesis” pipeline was implemented [5]. A single, highly active microservice trace was mathematically transformed into three distinct profiles: Steady, Intermediate, and Bursty [5]. Additionally, abstract Normalized

Compute Units (NCUs) were deterministically mapped to Real CPU Cores to ground the evaluation in physical hardware constraints [5].

3 Baseline Implementation

To rigorously quantify the problem, two industry-standard baselines were implemented and evaluated against the “Bursty” workload.

3.1 Policy Baseline: Reactive Kubernetes HPA

A simulation of the Kubernetes HPA was constructed using standard reactive threshold logic: scale up if CPU utilization exceeds 80%, and scale down if it falls below 50% [5]. Over a 30-day period, this reactive policy resulted in 2,107 distinct SLO violations (Figure 3) [5].

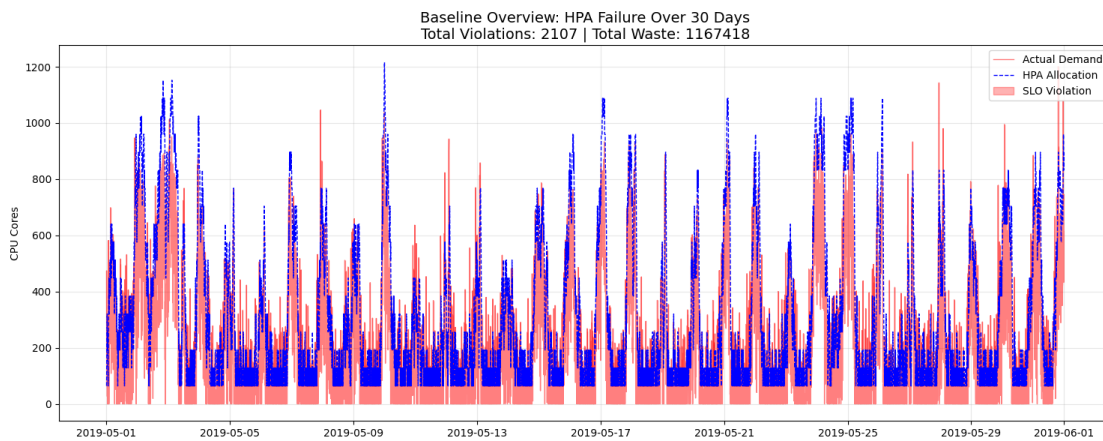


Figure 3: Baseline overview of Kubernetes HPA performance over 30 days, showing widespread SLO violations.

A zoom-in analysis (Figure 4) exposed a critical “Reaction Gap,” confirming that threshold-based scaling consistently allocates resources *after* the demand spike has already occurred [5].

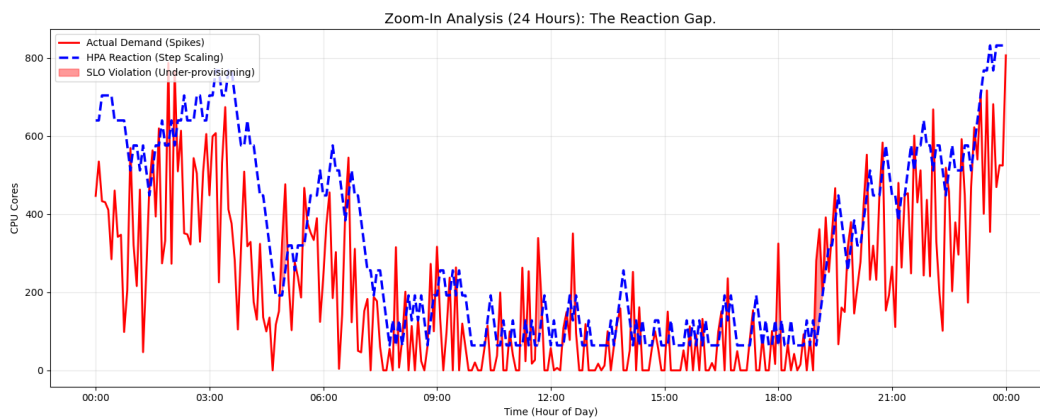


Figure 4: A 24-hour zoom-in highlighting the ‘Reaction Gap’ where step-scaling fails to catch sudden demand spikes.

3.2 Prediction Baseline: ARIMA vs. ML Screening

To establish a statistical performance benchmark, an ARIMA (5, 1, 0) model was initially implemented. As seen in Figure 5, the ARIMA model produced a flat-line prediction (MAE: 131.86, RMSE: 146.14), completely failing to anticipate sudden demand spikes because it relies on historical averages [5].

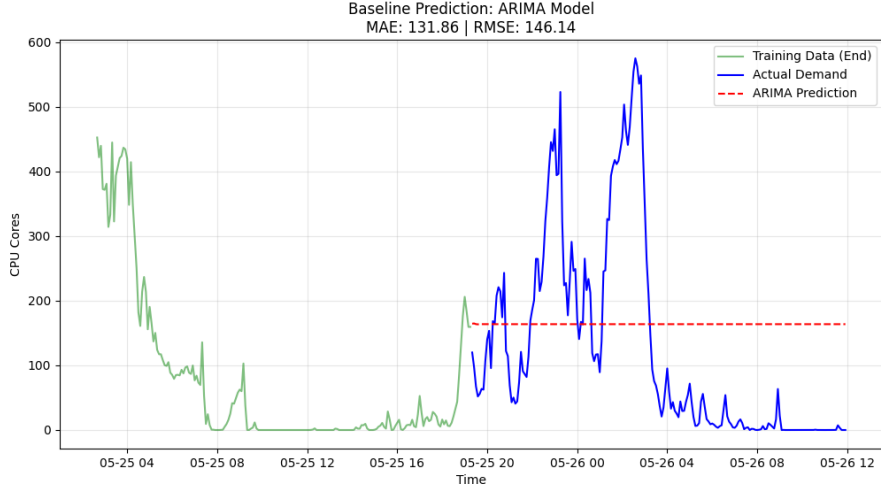


Figure 5: The ARIMA statistical baseline, demonstrating a failure to predict the onset of bursty workload phases.

To improve forecasting, an automated screening framework was developed in Google Colab to benchmark algorithms [6]. The results demonstrated that a simple Linear Regression model utilizing short-term “Lag Features” ($t-1$, $t-2$, $t-3$) successfully tracked actual demand (Figure 6), outperforming both the ARIMA baseline and Random Forest ensembles with an improved MAE of 97.46 [6]. Consequently, this Linear Regression model will serve as the forecasting feature generator for the RL agent [6].

4 Prototype Design: Formal Problem Formulation

The autoscaling problem is formally defined as a Markov Decision Process (MDP) designed for Constrained Optimization [6]. The objective is to minimize total cost, balancing resource usage against reliability penalties:

$$\min C_{total} = \alpha \cdot (N_t \cdot Cost_{node}) + \beta \cdot \max(0, D_t - C_t) \quad (1)$$

Where N_t is the number of active nodes, D_t is actual demand, C_t is total capacity, and the penalty weight (β) is set significantly higher than the cost weight (α) to prioritize reliability over pure cost savings [6].

The environment enforces the following physical constraints:

- **Scaling Rate Limit:** $|N_t - N_{t-1}| \leq 1$. The agent may only add or remove a single node per step, mimicking real-world autoscaler damping mechanisms and preventing instant jumps [6].
- **Capacity Limits:** $N_{min} \leq N_t \leq N_{max}$. The system cannot scale below 1 or exceed hardware limits [6].

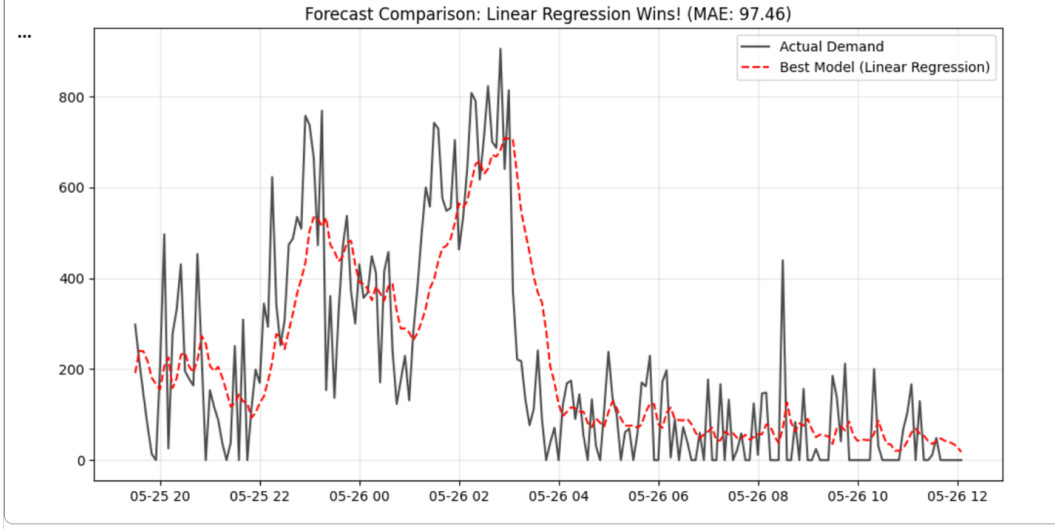


Figure 6: Linear Regression forecast comparison, showing superior tracking of non-linear bursts using lag features.

5 Experiment Plan & Reproducibility Harness

To transition from data analysis to systems engineering, a custom trace-driven simulator (**CloudAutoscalingEnv**) was built, adhering to the OpenAI Gym API standards [7]. This testbed ensures rigorous evaluation and absolute reproducibility for future RL training.

5.1 Predictor-to-Policy Loop

The previously validated Linear Regression model was embedded directly into the environment [7]. At each time step, the state space returns `[current_demand, current_replicas, forecast_demand]`, giving the policy an immediate look-ahead capability for proactive decision-making [7].

5.2 Reproducibility & Initial Verification

To adhere to systems research standards, global configurations were locked, including a fixed random seed (42) [7]. To validate the environment’s reward logic and physics, a rule-based Heuristic Agent was executed through the 30-day trace [7].

As shown in Figure 7, the heuristic agent successfully enforced the step-wise scaling constraints but still yielded a total of 2,462 SLO violations [7]. This establishes a concrete, mathematically proven baseline score that the proposed RL agent must minimize [7].

6 Next Steps

With the baseline pipeline complete and the experiment harness validated, the immediate next steps involve algorithm integration. Utilizing the **Stable-Baselines3** library, a Proximal Policy Optimization (PPO) agent will be instantiated and trained within the simulator for 100,000+ timesteps [7]. The fully trained policy will then be evaluated against the heuristic baseline to quantify improvements in reliability.

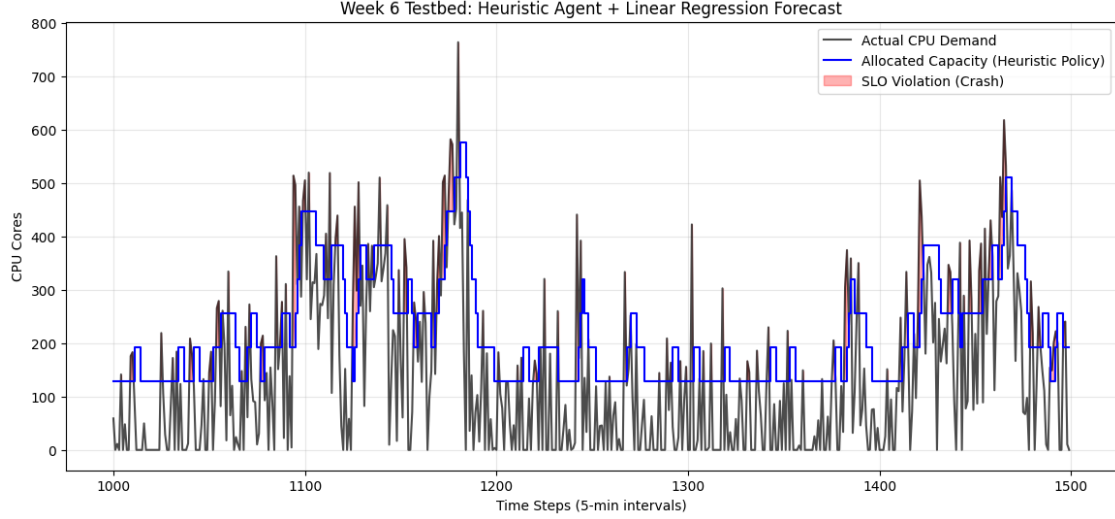


Figure 7: Simulation of the Week 6 testbed running a Heuristic Agent linked to the Linear Regression forecast.

7 Artifacts & Reproducibility

To ensure strict adherence to the project’s reproducibility standards, the experimental harness and baselines have been packaged with the following guarantees:

- **(i) Version Control:** All source code, exploratory data analysis scripts, and simulation environments are maintained in a version-controlled repository: <https://github.com/scr200/CP395-Research-Project>.
- **(ii) Environment Specification:** A `requirements.txt` file is provided in the repository root to ensure exact dependency matching (including `gym`, `scikit-learn`, and `statsmodels`).
- **(iii) Configurations and Seeds:** Determinism is enforced across all experiments. Data synthesis utilizes fixed seeds (e.g., `np.random.seed(101)` for noise injection), and the `CloudAutoscalingEnv` simulator is locked to `SEED = 42`.
- **(iv) Run Scripts:** Execution commands for each specific baseline (e.g., `baseline_policy.py`, `baseline_arima.py`, `W6_simulation.py`) are documented sequentially in the repository’s `README.md`.
- **(v) Experiment Logs:** The outputs of the automated model screening (e.g., `model_leaderboard.csv`) and step-by-step simulator tracking history are logged and linked to the configurations that generated them.

References

- [1] Wilkes, J., et al. (2020). *Google cluster-usage traces v3*. Google. Available at: <https://github.com/google/cluster-data>.
- [2] Rahemtulla, S. (2026). *CP395 Weekly Report 1*.

- [3] Rahemtulla, S. (2026). *CP395 Weekly Report 2*.
- [4] Rahemtulla, S. (2026). *CP395 Weekly Report 3*.
- [5] Rahemtulla, S. (2026). *CP395 Weekly Report 4*.
- [6] Rahemtulla, S. (2026). *CP395 Weekly Report 5*.
- [7] Rahemtulla, S. (2026). *CP395 Weekly Report 6*.